Spring Framework

Features of Spring framework:

IoC inversion of control container: in a normal condition or without spring framework we would need to create an instance(object) of the class and inject it where it should, manage it. This creates tight coupling. However the inversion of control allows spring boot using @Autowired to create an object for us, inject where its needed.

MVC: Model, View, control architecture is supported to separate data, http requests and the app logic.

Data access framework: it helps simplify and organize database connections.

Aspect Oriented Programming: AOP separates cross-cutting concerns from business logic.

A java bean is an object managed its lifecycle by spring container. The container creates and manages the bean. This bean can be wired using spring xml configuration, or spring boot annotations.

@configuration to configure a class must be non-final and public

@Bean to configure a bean and it must be not private and not final

A @Component is a class that becomes a Spring-managed bean directly, while a @Configuration class is also a bean but primarily serves as a factory for defining other beans.

@Autowired actually means

“Spring, resolve and inject the required bean(s) here.”

It can be applied to:

- Constructors, Fields, Setter methods

In the app configuration class when we define beans using methods we annotate those methods with @Bean, and this bean that is returned from the method is named the same as the method name but if we wrote @Bean(“name”) now the spring boot will name that bean as name.

Dependency injection mostly means bean injection.

Constructor injection is when the dependency or the java spring bean is passed into another bean or class's constructor

@Qualifier("primary") is a way that we use to prioritize or name our beans in case we have 2 beans of the same type, sometimes we would need to @Autowired them but our spring container does not know which one, so qualifier is how we tell the spring container which one it is.

Field injection is declaring a dependency or another bean of java spring and declaring it into another, directly (very discouraged since it makes the testing harder)

method injection in this case it to declare a method inside our class and inside this method we make an instance of our dependencies and @autowired it.

Setter injection works the same but the method is called set, like normal setters in normal OOP.

The bean scope is when is it available during the project lifecycle to interact with other beans, and that is by default is a singleton: only one instance of the bean.

Bean scopes: Singleton Prototype Request, Session, Application, Web socket.

@Scope("Prototype") or any other scope to help us define the scope for the bean. After @Bean

Environment abstraction bean environment,

@Profile is a conditional switch that determines whether a bean is loaded into the Spring container based on the active environment. Whether its testing, development or production

@Value("Actual value") is used to inject value into a variable, whether its hardcoded or comes from another environment or class. Or class path, or file or variable.

Some best practices for spring boot:

Split configuration classes based on the architecture. Into multiple classes

Spring boot in a nutshell is a less configuration spring.

Spring Boot simplifies Spring by making applications **self-contained, auto-configured, and easy to run**, mainly through standalone execution, embedded servers, and starter dependencies.

Spring REST

Representational stateless transfer, a software architecture to make web services to which usually are called RESTFUL APIs.

The main idea is to treat the network components as objects that can be accessed using standard http methods

It's a client server architecture they can work together but each one of them works and handles data independently.

Its stateless meaning the server does not store the requests only executes them

Its layered cacheable system

we have linking of resources

we have standard methods GET, POST, PUT, DELETE

the resources that hold the data should be plural nouns for instance

GET / accounts → retrieves all the accounts

GET/ accounts/1 → retrieves the account with the id 1

In case of nested resources payments inside the accounts it should be like this

GET/accounts/1/payments/2

This returns the payment of id 2 inside the account of id 1

We use http messages GET, POST, PUT, DELETE,PATCH,OPTIONS,HEAD to specify what should be done

GET method

@GetMapping(“/users/{id}”)

Public void printProduct(){}

This annotation is triggered aka mapped into the function its on once the http request of get/users/{id} is accessed it just fetches data from the server and show it to the user its very safe

POST

@PostMapping(“/users”)

Post is used to create a new resource in the database

PUT

@PutMapping

```
@PutMapping("/users/{id}")
public User updateUser(@PathVariable Long id, @RequestBody User user) {
   user.setId(id);
   return userService.update(user);
}
```

Update an existing resource by replacing it completely.

How it behaves:

- Client sends full updated object
- Existing data is completely replaced
- Missing fields may be overwritten (often set to null)

DELETE

```
@DeleteMapping("/users/{id}")

public void deleteUser(@PathVariable Long id) {

   userService.delete(id);

}
```

Is used to remove a resource from the databse

PUT

```
@PutMapping(“/users”)
```

```
@PatchMapping("/users/{id}")
public User patchUser(@PathVariable Long id, @RequestBody Map<String, Object> updates) {
   return userService.partialUpdate(id, updates);
}
```

Update only specific fields of a resource (not the whole object).

Real-life analogy:
Changing just your email address, not your entire profile.

OPTIONS — Discover Supported Methods

```
@RequestMapping(value = "/users", method = RequestMethod.OPTIONS)
public ResponseEntity<?> options() {
   return ResponseEntity.ok()
      .header("Allow", "GET, POST, PUT, DELETE, PATCH, OPTIONS")
      .build();
}
```

Purpose:
Ask the server what operations are allowed on a resource.

HEAD — Retrieve Headers Only

```
@RequestMapping(value = "/users/{id}", method = RequestMethod.HEAD)
public ResponseEntity<Void> headUser(@PathVariable Long id) {
   if (userService.exists(id)) {
      return ResponseEntity.ok().build();
   } else {
      return ResponseEntity.notFound().build();
   }
}
```

Purpose:
Same as GET, but returns only headers (no response body).

Response Status code

First digit defines the class of the response while the last 2 digits they don't have any specific

1xx: Informational

2xx: Success…. 200 OK -> standard response for success, 201 CREATED -> A new resource was created from the post method, 204 no content DELETE PUT POST

3xx: Redirection, 304 NOT Modified → for caching, if the current version matches the version in the client cache we get this message from the server and no more content. We just use the local version of the request.

4xx: client error, 400 bad request, the server was not able to understand the request because of invalid syntax. 401 unauthorized -> means requires user authentication. 403 forbidden the client does not have the necessary authorization for the certain resources.

5xx: server error, 500 server error when the server has an undefined issue and no more information is known or provided. 503 server unavailable, which means that the server is down because its overloaded or under maintenance.

What is a @RestController

A @RestController is a Spring annotation used to define a controller that handles HTTP requests and returns data directly in the response body. A controller processes incoming HTTP requests, executes application logic, and produces a response. Unlike a regular @Controller, which typically returns a view (HTML page), a @RestController returns raw data such as JSON or XML.

@ResponseStatus

What is @ResponseStatus?

@ResponseStatus is an annotation used to:

Manually set the HTTP status code of a response

```
@PostMapping("/users")
@ResponseStatus(HttpStatus.CREATED)
public User createUser(@RequestBody User user) {
return user;
}
```

@RequestBody

Take the HTTP request body (usually JSON) and convert it into a Java object. @RequestBody is an annotation used in Spring MVC / Spring Boot to indicate:

Take the raw HTTP request body, deserialize it (usually from JSON), and bind it to a Java object so it can be used in backend logic

Example

```
@PostMapping("/user/post")
@ResponseStatus(HttpStatus.CREATED)
@Autowired → incorrect there is no dependency injection to be done
public String printUser( @RequestBody Order order){
   return "welcome the order details are " + order.toString();
}
```

To explain the dynamics, here we have a post (create new user or data), and we just take the data from the user and printing it no logic is happening in the backend, the @RequestBody is the

focus. The request body tells the Spring boot that we will be passing data in the http request body and this data should be changed from JSON into JAVA object called order

After collecting this data we should then inside the order class map our json into java variables.

Without @RequestBody spring boot will try to deserialize the data automatically

To map the data into java variables we should use @JsonProperty if the variable names are different from the JSON names of the corresponding variable

Record class

Records are final by default and all their fields are also final

A concise way to declare a class whose main purpose is to hold data, with built-in immutability and automatically generated methods.

Example from the online course: when we were mapping the JSON data from the request body into the class we were using a class called order, inside this class we mapped the json variables on the variables of the class, then declared setters and getters.

A record class declared as public record order(variable1, variable2, variable3){}

The record creates setters and getters and toString() automatically and maps the json data on our record.

A record can implement an interface but cannot extend a class.

```
Public record OrderRequest(
@JsonProperty(“customer_name”) String customerName,
@JsonProperty(“product_name”) String productName,
int quantity
) {}
```

@pathvariable

@PathVariable is used to capture dynamic values from the URL path and pass them into a controller method.

```
@GetMapping("/orders/{id}")

public String getOrder(@PathVariable int id) {

   return "Order ID: " + id;}
```

@RequestParam

Variable as request parameter

Request dispatching

```
@GetMapping("/orders")
public String getOrder(
   @RequestParam int id,
   @RequestParam String status
) {
   return id + " " + status;
}
```

A method name should be unique even if it accepts different path variable or parameters still it should hold a unique name, for each method so for example

We can have 2 method called /hello but for different methods so one can be get and the other post

But we cannot have 2 hello methods even if the first one is accepting path variable and the second not since both are GET

Data source configuration

Spring:

    Datasource:

        url:

    username:

    password:

    driver-class-name: for this we need to add the database dependency of mysql

Persistent entity for the database

In the context of Java Spring Boot and Spring Data JPA, a persistent entity is a central concept that represents how Java objects are mapped to database tables.

Where each java object instance is stored as row in this persistent entity

Database row → loaded back as a Java object

JPA defines the rules for mapping Java objects to relational database tables.

Hibernate (usually used under Spring Boot) is the engine that performs the actual work.

Spring Data JPA simplifies repository access (CRUD operations) so you don't write SQL for most operations.

@Entity

Is how you define a java class to be a persistent JPA entity

@Table(name = "table_name") this step is optional, to give a name for the table.

@Column (optional)

Used to customize column details:

```
@Column(name = "email_address", nullable = false)
private String email;
```

```
@Column(
name = "email_address",
nullable = false,
unique = true,
length = 150
) all of the information here is optional
```

```
@Table(
  name = "users",
  uniqueConstraints = {
    @UniqueConstraint(columnNames = {"email", "tenant_id"})
  }
)
```

this is very strong and it forces the table to have only one email and one tenant_id per user name

@Entity @Table we have to have an empty constructor—we need to specify a primary key each time we create an entity

@ Id

is one of the most fundamental annotations. It defines the primary key of a persistent entity.

Without it, JPA cannot uniquely identify rows in the database, and the entity is not valid for persistence.

@GeneratedValue

Without @GeneratedValue, you must assign IDs manually:

user.setId(1L);

but on using @GeneratedValue the Spring JPA assigns a value to our primary key, mostly surrogate key

@GeneratedValue(strategy = GenerationType.X)

Here we can get more specific about how the automatic values will be assigned to our primary key. The best so far is AUTO

.IDENTITY is the equivalent of usin AUTO_INCREMENT in SQL

```
Jpa:
    Hibernate:
        Ddl-auto: create
    Show-sql: true
    Properties:
        Hibernate:
            Format-sql: true
        Database-paltform
```

jpaRepository Interface

The hierarchy is:

Repository (marker interface)
↓
CrudRepository
↓
PagingAndSortingRepository
↓
JpaRepository

JpaRepository is the **most feature-complete interface in this chain**. It will give us more control, over our repo and the most optioned layer. It a generic interface.

It has all the CRUD operations: C**rudRepository**

Adds basic CRUD:

save()
findById()
findAll()
delete()
count()
existsById()

flush()

saveAndFlush()

deleteInBatch()

findAll(Specification)

userRepository.save(user);

what happens here is that JpaRepository checks whether the user entity object has an already Id if id == null then it creates a row using persist(), if its untracked and exists meaning that id != null, it calls for merge(). If it exists and is tracked nothing happens.

Optional<User> user = userRepository.findById(1L); is equivalent to the sql of

SELECT * FROM TABLE WHERE USER_ID = 1

@query